

Cyber Security Internship Project Report

Name: Muhammad Talha

Internship Domain: Cyber Security

Email: mtalha27709@gmail.com

Phone Number: 03316216291

Organization: Arch Technologies

Submission: Month 1

PROJECT TITLE

Task 1: Basic Network Sniffer

Task 2: Key Logging Simulation (Safe Environment)

INTRODUCTION

This report includes two cybersecurity projects completed as part of an internship at Arch Technologies.

The objective of these tasks was to gain hands-on experience in:

- Network traffic monitoring and packet analysis
- Understanding keystroke logging techniques in a safe simulated environment

These projects helped improve practical knowledge of cybersecurity tools, Python programming, and threat analysis concepts.

TASK 1: BASIC NETWORK SNIFFER

Objective

The objective of this project was to develop a basic network packet sniffer using Python and Scapy to capture and analyze live network traffic.

Tools Used

- Python 3
- Scapy Library
- Windows PowerShell
- VS Code

Methodology

1. Installed Python and required Scapy library
2. Created a Python script to capture network packets

3. Used Scapy sniff() function for live packet capture
4. Extracted packet information such as:
 - Source IP address
 - Destination IP address
 - Protocol type
 - Packet length
5. Analyzed real-time network traffic flow

Observations

- TCP, UDP, and ICMP packets were observed during testing
- Source and destination IP addresses were successfully captured
- Live network traffic monitoring was achieved

Challenges

- Installing and configuring Scapy properly
- Running Python script with administrator privileges
- Understanding packet structure and layers

Screenshots

This screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays the project structure with files 'swsniffer.py' and 'update_snsiffer.py'. The main editor window shows the 'swsniffer.py' file with the following code:

```
1 from scapy.all import sniff
2
3 def packet_callback(packet):
4     print(packet.summary())
5
6 print("Starting packet capture...\n")
7
8 sniff(prn=packet_callback, count=10)
9
10
```

The bottom panel shows the TERMINAL with the command prompt at 'C:\Users\TALHA\Desktop\Internships\codealpha>'.

This screenshot shows the same VS Code editor interface, but now the script is running. The terminal output shows the command prompt executing the script:

```
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha>
PS C:\Users\TALHA\Desktop\Internships\codealpha> & C:\Users\TALHA\AppData\Local\Programs\Python\Python314\python.exe c:/Users/TALHA/Desktop/Internships/codealpha/swsniffer.py
Starting packet capture...
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https PA / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https A / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https A / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https A / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https A / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https A / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https A / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https A / Raw
Ether / IP / TCP 192.168.0.106:61191 > 51.11.192.49:https PA / Raw
Ether / IP / UDP / DNS Qry b'mobile.events.data.microsoft.com.'
PS C:\Users\TALHA\Desktop\Internships\codealpha>
```

Code Section (Task 1)

```
from scapy.all import sniff
def packet_callback(packet):
    print(packet.summary())
print("Starting packet capture...\n")
sniff(prn=packet_callback, count=10)
```

This screenshot shows the initial setup of a packet sniffer script in VS Code. The Explorer panel on the left shows the project structure with files 'Welcome', 'swiffer.py', and 'update_sniffer.py'. The main editor displays the 'update_sniffer.py' file with the following code:

```
1 from scapy.all import *
2
3 def packet_callback(packet):
4     if IP in packet:
5         print(f"""
6 Source IP: {packet[IP].src}
7 Destination IP: {packet[IP].dst}
8 Protocol: {packet[IP].proto}
9 Packet Length: {len(packet)}
10 -----
11 """)
12
13 print("Sniffing packets...\n")
14
15 sniff(prn=packet_callback, count=15)
16
17
18
```

The bottom panel shows the terminal with the command prompt at the directory 'C:\Users\TALHA\Desktop\Internships\codealpha'. The status bar at the bottom indicates the file is at line 18, column 1, with 4 spaces, UTF-8 encoding, and CRLF line endings.

This screenshot shows the execution of the packet sniffer script. The terminal output displays the following information for three captured packets:

```
PS C:\Users\TALHA\Desktop\Internships\codealpha> & C:\Users\TALHA\AppData\Local\Programs\Python\Python314\python.exe c:/Use
rs/TALHA/Desktop/Internships/codealpha/update_sniffer.py
Sniffing packets...

Source IP: 140.82.114.22
Destination IP: 192.168.0.106
Protocol: 6
Packet Length: 66
-----

Source IP: 192.168.0.106
Destination IP: 140.82.114.22
Protocol: 6
Packet Length: 54
-----

Source IP: 192.168.0.106
Destination IP: 140.82.114.22
Protocol: 6
Packet Length: 571
-----

Source IP: 140.82.114.22
Destination IP: 192.168.0.106
Protocol: 6
Packet Length: 1454
-----
```

The status bar at the bottom indicates the file is at line 18, column 1, with 4 spaces, UTF-8 encoding, and CRLF line endings.

Code Section (Task 1)

```
from scapy.all import *
def packet_callback(packet):
    if IP in packet:
```

```
print(f"""
Source IP: {packet[IP].src}

Destination IP: {packet[IP].dst}

Protocol: {packet[IP].proto}

Packet Length: {len(packet)}

-----

""")
print("Sniffing packets...\n")
sniff(prn=packet_callback, count=15)
```

Conclusion (Task 1)

The project successfully demonstrated basic packet sniffing and provided hands-on experience with network traffic analysis using Python.

TASK 2: KEY LOGGING SIMULATION (SAFE)

Objective

The objective of this project was to simulate a basic key logging mechanism in a controlled environment to understand how keystroke logging works and its associated security risks.

Tools Used

- Python 3
- Tkinter Library

- VS Code

Methodology

1. Created a simple GUI application using Tkinter
2. Added a text input box for user input
3. Captured user-typed text manually
4. Stored input data in a local text file (log.txt)
5. Recorded timestamp with each entry

Observations

- User input was successfully captured
- Data was stored in a local log file
- Timestamp tracking worked correctly

Security Risks

- Unauthorized data theft
- Password leakage
- Privacy violations
- Potential malware usage in real-world attacks

Prevention Methods

- Use updated antivirus software
- Avoid unknown applications
- Enable firewall protection
- Do not install untrusted software

Screenshots

This screenshot shows the first 38 lines of the `key_logging.py` script in VS Code. The Explorer sidebar on the left shows the project structure with `key_logging.py` and `log.txt` under the `KEY_LOGGING` folder. The main editor window displays the following code:

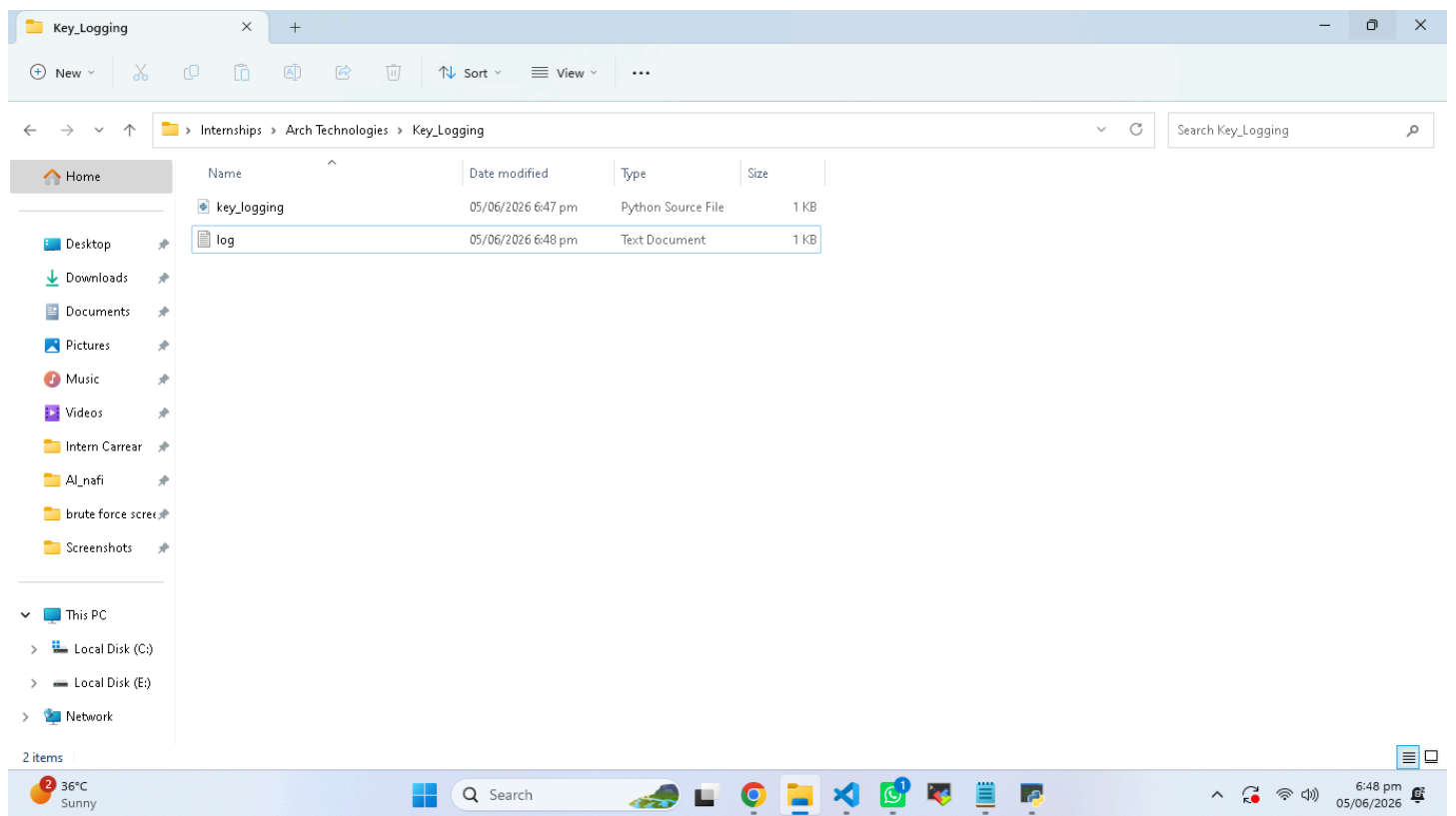
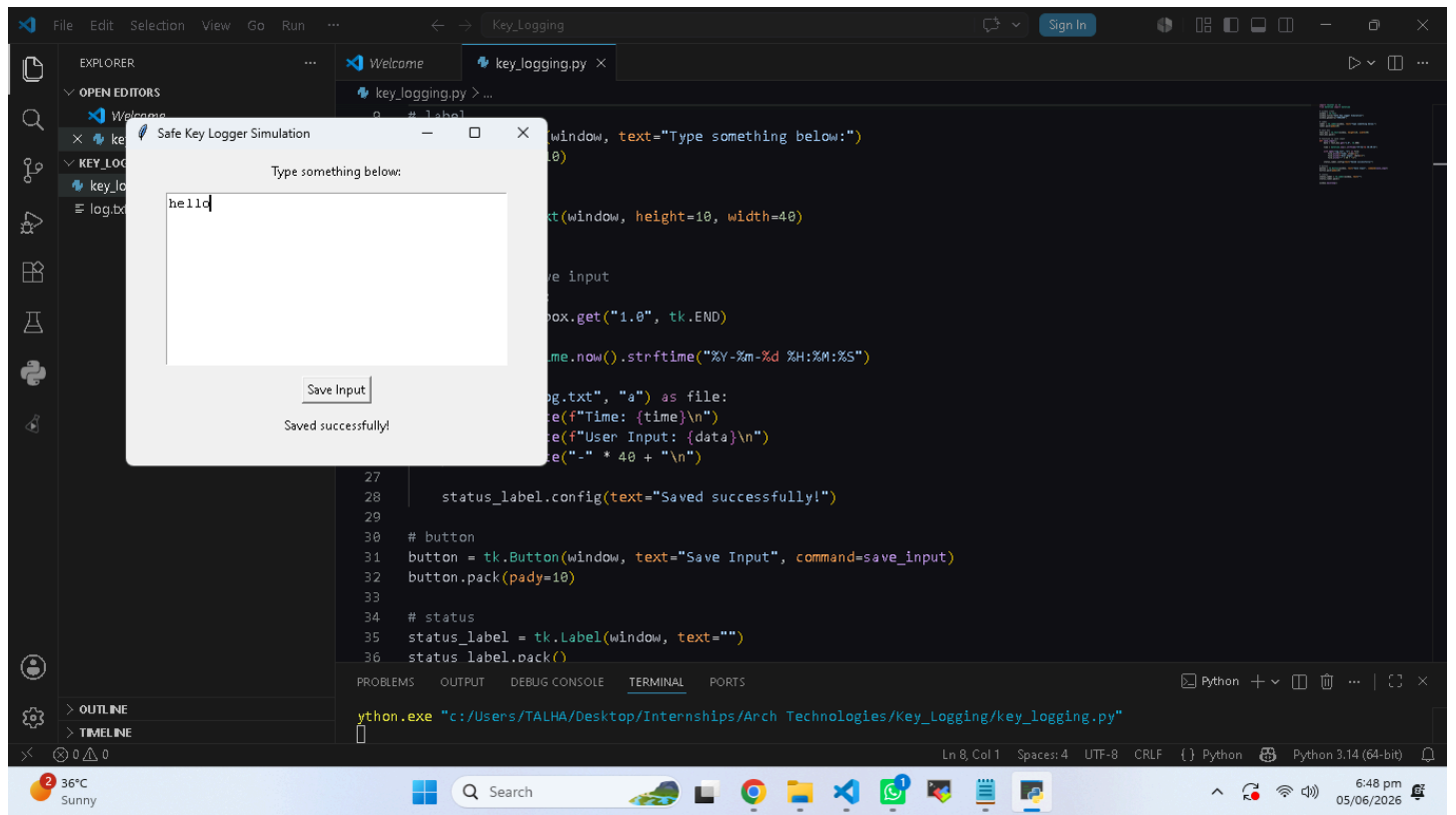
```
9 # Label
10 label = tk.Label(window, text="Type something below:")
11 label.pack(pady=10)
12
13 # text box
14 text_box = tk.Text(window, height=10, width=40)
15 text_box.pack()
16
17 # function to save input
18 def save_input():
19     data = text_box.get("1.0", tk.END)
20
21     time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
22
23     with open("log.txt", "a") as file:
24         file.write(f"Time: {time}\n")
25         file.write(f"User Input: {data}\n")
26         file.write("-" * 40 + "\n")
27
28     status_label.config(text="Saved successfully!")
29
30 # button
31 button = tk.Button(window, text="Save Input", command=save_input)
32 button.pack(pady=10)
33
34 # status
35 status_label = tk.Label(window, text="")
36 status_label.pack()
37
38 window.mainloop()
```

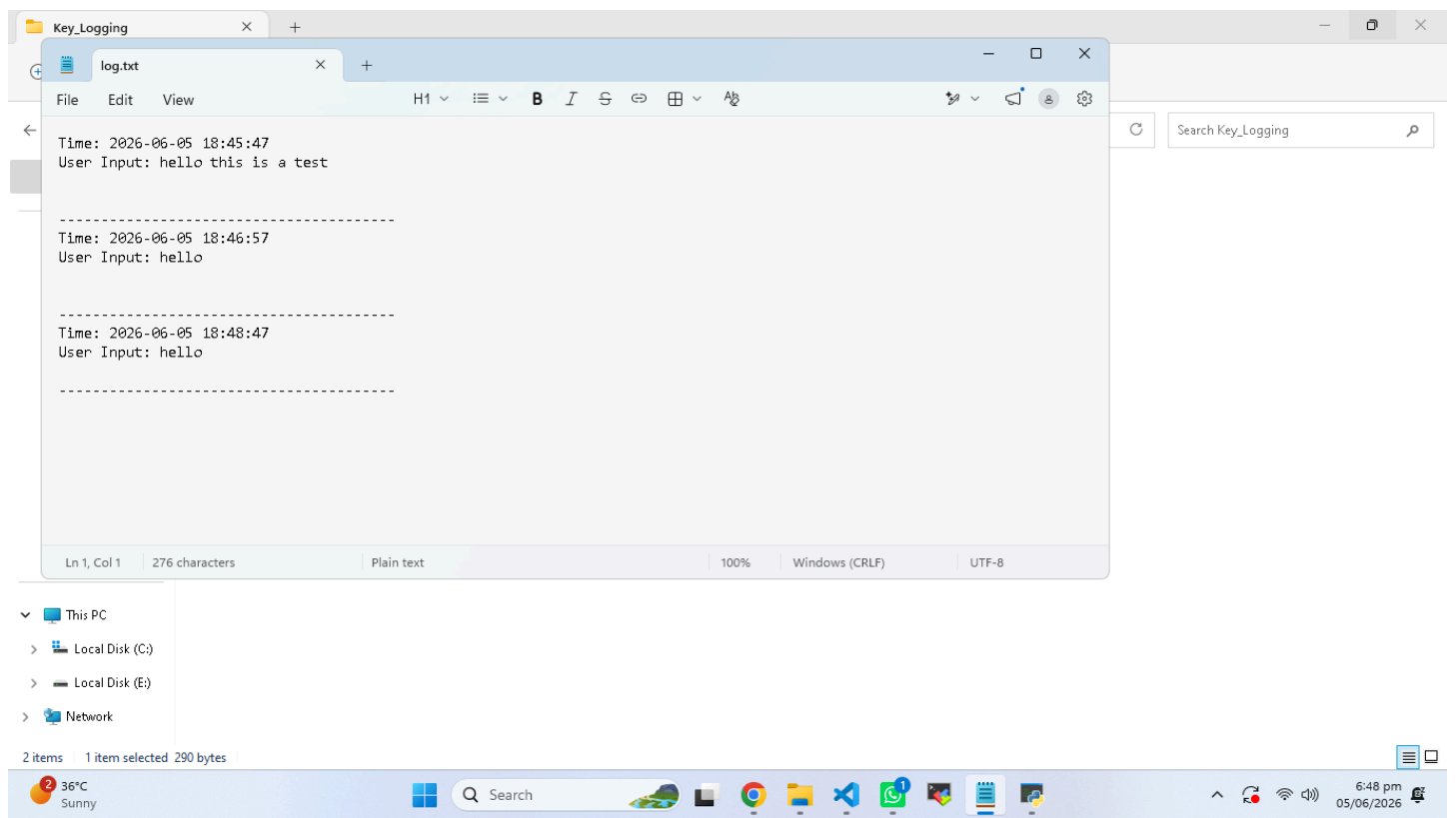
The status bar at the bottom indicates the cursor is at line 8, column 1, with 4 spaces, UTF-8 encoding, and CRLF line endings. The system tray shows a temperature of 36°C and the date 05/06/2026.

This screenshot shows the continuation of the `key_logging.py` script, starting from line 1. The Explorer sidebar remains the same. The main editor window displays the following code:

```
1 import tkinter as tk
2 from datetime import datetime
3
4 # window creat
5 window = tk.Tk()
6 window.title("Safe Key Logger Simulation")
7 window.geometry("400x300")
8
9 # Label
10 label = tk.Label(window, text="Type something below:")
11 label.pack(pady=10)
12
13 # text box
14 text_box = tk.Text(window, height=10, width=40)
15 text_box.pack()
16
17 # function to save input
18 def save_input():
19     data = text_box.get("1.0", tk.END)
20
21     time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
22
23     with open("log.txt", "a") as file:
24         file.write(f"Time: {time}\n")
25         file.write(f"User Input: {data}\n")
26         file.write("-" * 40 + "\n")
27
28     status_label.config(text="Saved successfully!")
29
30 # button
31 button = tk.Button(window, text="Save Input", command=save_input)
32 button.pack(pady=10)
```

The status bar at the bottom indicates the cursor is at line 38, column 18, with 4 spaces, UTF-8 encoding, and CRLF line endings. The system tray shows a temperature of 36°C and the date 05/06/2026.





Code Section (Task 2)

```
import tkinter as tk

from datetime import datetime
# window creat

window = tk.Tk()

window.title("Safe Key Logger Simulation")

window.geometry("400x300")
# label

label = tk.Label(window, text="Type something below:")

label.pack(pady=10)
# text box

text_box = tk.Text(window, height=10, width=40)

text_box.pack()
# function to save input

def save_input():
```

```

data = text_box.get("1.0", tk.END)
time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
with open("log.txt", "a") as file:

    file.write(f"Time: {time}\n")

    file.write(f"User Input: {data}\n")

    file.write("-" * 40 + "\n")
    status_label.config(text="Saved successfully!")
# button

button = tk.Button(window, text="Save Input", command=save_input)

button.pack(pady=10)
# status

status_label = tk.Label(window, text="")

status_label.pack()
window.mainloop()

```

Conclusion (Task 2)

The project demonstrated a safe simulation of key logging and helped in understanding how keystroke monitoring works and its cybersecurity implications.

FINAL CONCLUSION (OVERALL)

Both projects provided practical exposure to cybersecurity fundamentals including network monitoring and keystroke logging simulation. These tasks improved understanding of how attackers exploit systems and how such threats can be mitigated.